

DAY 7 — DETAILED TO-DO

Backend Reliability + Hashing/Intervals + Production Repository Hardening

Textbook-style execution guide based on the Day 7 planning conversation.

DAY 7 — Backend Reliability + Hashing/Intervals + Production Repository Hardening

Core objective

Take the backend you have already built and start thinking like a production SWE: database-backed API → correctness under failure → concurrency/reliability → stronger tests → system-design reasoning.

Day 7 is deliberately not another feature-heavy day.

You have now established:

```
Python
↓
FastAPI
↓
Pydantic
↓
Service Layer
↓
Repository Layer
↓
PostgreSQL
↓
Migrations
↓
Integration Tests
```

Your Project 1 repository already reflects this progression.

The next step is to make that foundation robust, while DSA moves away from the Prefix Sum material you just completed.

DAY 7 — TIME BUDGET

Target: ~3 hours

Area	Time	Priority
DSA — Intervals + Hashing	50 min	■
Backend reliability theory	30 min	■
Project code review + hardening	60 min	■
Testing / failure cases	25 min	■
System design	10 min	■
Documentation / Git	5 min	■

70% build / 20% study / 10% documentation still applies.

PART 1 — FIRST 10 MINUTES

Establish the Day 7 Baseline

Do not immediately start coding.

You need evidence of the current state first.

1.1 Project 1

Run:

```
git status
git log --oneline --decorate -n 10
```

Then:

```
pytest -q
```

or your repository's established pytest command.

Your README currently documents:

```
tests/test_api.py
tests/test_postgres_integration.py
```

and the PostgreSQL integration suite is already part of the project.

Record:

```
Current commit:
Current branch:
Tests passed:
Tests failed:
Warnings:
Working tree:
```

Do not assume a clean test run simply because the README says the tests exist.

1.2 Inspect the architecture

Read these files before changing anything:

```
src/main.py
src/api/document_routes.py
src/services/document_service.py
src/repositories/document_repository.py
src/db/connection.py
src/database.py
src/models/pydantic_model.py
src/data_classes/document_class.py
src/exceptions/document_exceptions.py
tests/test_api.py
tests/test_postgres_integration.py
```

These are explicitly part of the current repository structure.

Your goal is to answer:

“Can I explain the complete request lifecycle without looking at the code?”

```
HTTP request
↓
FastAPI route
↓
Pydantic validation
↓
DocumentService
↓
DocumentRepository
↓
database_connection()
↓
PostgreSQL
↓
result
↓
domain/service response
↓
HTTP response
```

If you cannot explain one arrow, study that arrow before modifying it.

PART 2 — DAY 7 DSA

Hashing + Intervals

You have already worked through:

- Two Pointers
- Sliding Window
- Prefix Sum
- Prefix Sum + Hashing

Therefore do not solve another Prefix Sum problem today.

Today's objective is to broaden your pattern vocabulary.

DSA PROBLEM 1 — Group Anagrams

Pattern: Hashing + canonical representation

You need to recognize that different strings can map to the same logical signature.

```
eat
tea
ate
```

all belong to the same group.

```
word
↓
signature
↓
hash map
↓
group
```

Study questions

- What makes two strings anagrams?
- What can serve as a canonical representation?
- What should the hash-map key be?
- What is the cost of sorting every word?
- Can character counting be used?

Required analysis

```
Problem
Pattern
Brute Force
Optimized
Why HashMap?
Key representation
Complexity
Edge cases
```

DSA PROBLEM 2 — Longest Consecutive Sequence

Pattern: Hash Set

This is an important Google-style pattern because the key is not immediately obvious.

```
numbers
↓
set
↓
identify sequence starts
↓
expand only from starts
```

The important insight is:

Don't sort if the problem can be solved using constant-time average membership checks.

Target:

$O(n)$

average time.

Interview follow-ups

- Why does sorting give $O(n \log n)$?
- Why can a hash set achieve expected $O(n)$?
- How do you identify the beginning of a sequence?
- Why don't you expand every number?

DSA PROBLEM 3 — Merge Intervals

Pattern: Sort + Greedy interval merging

```
[1, 3]
[2, 6]
[8, 10]
[9, 12]
```

becomes:

```
[1, 6]
[8, 12]
```

Core reasoning

Sort by:

start time

Then maintain:

current interval

and determine whether the next interval overlaps.

Interview questions

- Why do we sort?
- What invariant are we maintaining?
- When do two intervals overlap?
- What happens when they do not overlap?
- Why is the complexity $O(n \log n)$?

DSA PROBLEM 4 — OPTIONAL IF TIME PERMITS

Insert Interval

Only do this if the first three are genuinely understood.

Do not solve it mechanically.

Focus on:

before overlap
overlap region
after overlap

This is useful because it builds on Merge Intervals without simply repeating it.

DSA NOTE FORMAT

For each problem, record:

1. Problem
2. Pattern
3. Recognition clues
4. Brute force
5. Brute-force complexity
6. Optimized intuition
7. Invariant
8. Algorithm
9. Complexity
10. Edge cases
11. Common mistakes
12. Alternative approach
13. Google follow-ups

The most important addition today is:

Pattern recognition clues

"Need fast membership"
→ Hash Set

"Group equivalent objects"
→ Hash Map + canonical key

"Overlapping ranges"
→ Sort + Intervals

"Merge overlapping ranges"
→ Merge Intervals

This directly supports your DSA objective of pattern recognition.

PART 3 — BACKEND THEORY

Production Failure Thinking

30 minutes

You already implemented:

- connection boundary
- rollback on failure
- connection cleanup
- environment configuration
- migrations
- PostgreSQL repository

- integration tests

Now understand why those things exist.

3.1 What happens when PostgreSQL goes down?

```
FastAPI
↓
Service
↓
Repository
↓
PostgreSQL
```

Now imagine:

```
PostgreSQL
X
```

Ask: What should the API return?

```
database failure
↓
repository exception
↓
service?
↓
global exception handler
↓
HTTP response
```

You do not need to implement a complete resilience framework today.

But you must understand the failure boundary.

3.2 Connection lifecycle

Your current implementation explicitly has:

```
database_connection()
```

which opens a connection, yields it to the repository, rolls back on failure, and closes it in finally.

You should now understand:

Why finally?

Because cleanup must occur even when an exception happens.

```
connection = open()

try:
    yield connection
except:
    rollback()
    raise
finally:
    close()
```

Know why each part exists.

3.3 Rollback

Understand this failure:

```
operation A
↓
success
```

operation B
↓
failure

What should happen?

You need to understand:

COMMIT
ROLLBACK

and the difference between:

application exception

and:

database transaction failure

3.4 Connection pooling

You do not need to implement a sophisticated pool today.

But learn:

Why is opening a fresh database connection for every request potentially expensive?

Request
↓
Acquire connection from pool
↓
Query
↓
Release connection

versus:

Request
↓
Create TCP/database connection
↓
Query
↓
Close connection

Know the trade-off.

3.5 Concurrency

This is your first serious backend concurrency question.

Document A
version = 1.0

Two requests arrive:

Request A → update → 2.0
Request B → update → 3.0

What happens?

Potential problem:

lost update

You should understand the concept of:

Optimistic concurrency

```
UPDATE documents
SET version = ...
WHERE id = ...
AND version = expected_version
```

If zero rows are updated:

someone else changed the document

You don't have to implement optimistic locking today.

But you must understand why it exists.

PART 4 — PROJECT 1

Reliability Review

This is the major Day 7 build.

Do NOT add a new feature.

Instead:

Find weaknesses in the existing implementation and harden one or two of them.

This is deliberate.

You already have enough architecture to begin engineering for failure.

4.1 Review database failure behavior

Inspect:

```
src/db/connection.py
src/repositories/document_repository.py
src/main.py
```

Ask:

- What exception occurs when PostgreSQL is unreachable?
- Where does that exception go?
- What HTTP response does the client receive?
- Does the connection get closed?
- Does the transaction get rolled back?
- Could the API expose internal database details?

4.2 Review SQL safety

Inspect every SQL statement.

Verify parameters are passed separately rather than interpolated into SQL strings.

Bad:

```
f"SELECT * FROM documents WHERE id = '{document_id}'"
```

Good conceptually:

```
cursor.execute(
    "SELECT ... WHERE id = %s",
    (document_id,)
)
```

Your Day 7 goal is to understand:

SQL injection prevention is an application correctness and security requirement, not merely a database topic.

4.3 Review resource cleanup

For every database operation, verify:

```
connection opened
↓
cursor acquired
↓
query
↓
result
↓
cursor/connection cleaned up
```

Pay particular attention to exception paths.

4.4 Review repository responsibilities

```
Route
↓
Service
↓
Repository
↓
Database
```

Route should handle: HTTP concerns

Service should handle: business logic

Repository should handle: persistence

Database layer should handle: connection lifecycle/configuration

This separation should remain clean.

4.5 Find one architectural smell

You need to identify at least one real improvement opportunity.

Examples:

- duplicate logic
- unclear error translation
- poor naming
- too much repository responsibility
- too much service responsibility
- weak validation
- resource-management weakness
- test coupling
- configuration weakness

Do not invent a problem just to make a change.

If you find no meaningful issue, document:

“Reviewed X; no change required because Y.”

That is better engineering than unnecessary refactoring.

PART 5 — TESTING

Failure-Oriented Testing

25 minutes

Your current repository already has normal CRUD integration coverage.

Day 7 should move beyond:

happy path

into:

failure path

Test Category 1 — Invalid database configuration

Understand what happens when:

`DATABASE_URL`

is missing or unusable.

Your current README notes that placeholder DB values are ignored to avoid invalid runtime attempts. Verify that behavior against the actual implementation.

Test Category 2 — Database unavailable

PostgreSQL stopped
↓
GET /documents

Document the observed behavior.

If you can safely test it locally, do so.

Do not fake a test result.

Your project rule is:

Never fabricate metrics or results.

Test Category 3 — Missing resource

GET /documents/nonexistent
→ 404

and:

DELETE /documents/nonexistent
→ 404

The current README says these are covered.

Test Category 4 — Duplicate ID / integrity behavior

Investigate what happens when an insertion violates:

PRIMARY KEY

Do not immediately add custom behavior.

First:

1. reproduce
2. observe exception
3. understand exception type
4. decide where translation belongs
5. only then change code if justified

This is much closer to production debugging.

Test Category 5 — Update race reasoning

You don't need to create a complete concurrency test today.

Instead document:

Current behavior:
Potential race:
Why it happens:
Possible solution:

PART 6 — SYSTEM DESIGN

Document Service at 10× Scale

10 minutes

You have already considered the 1-million-document design in the Day 5 material.

Now take the actual implementation and scale it.

```
Client
↓
FastAPI
↓
Service
↓
Repository
↓
PostgreSQL
```

Ask: What breaks first at 10× traffic?

Think through:

API

```
Load Balancer
↓
FastAPI instances
```

Database

connection pool

Reads

indexes
read replicas
cache

Writes

primary database

Reliability

timeouts
retries
backpressure

health checks

Observability

latency
error rate
DB latency
connection pool saturation

Do not implement these today.

Just design them.

PART 7 — INTERVIEW PREPARATION

Day 7 interview questions should be based on what you actually built, not generic trivia.

Backend

- Why did you introduce a repository layer?
- Why is the database connection a separate boundary?
- Why shouldn't FastAPI routes execute SQL directly?
- How does a request travel through your application?
- What happens when PostgreSQL is unavailable?
- How are connections cleaned up?
- Why use finally?
- Why rollback on database failure?
- What is connection pooling?
- What is a lost update?
- How could you prevent concurrent updates from overwriting one another?
- What is optimistic concurrency control?
- How would you make the document API horizontally scalable?
- What would you cache?
- Where would you add observability?

Database

- Why PostgreSQL instead of an in-memory dictionary?
- Why is id a primary key?
- What indexes would you add?
- What is the trade-off of adding indexes?
- What is a transaction?
- What is isolation?
- What happens when a transaction fails?
- What happens when two requests update the same row?
- What is a connection pool?

- How would you scale PostgreSQL?

DSA

- How do you recognize a Hash Set problem?
- Why does Longest Consecutive Sequence achieve expected $O(n)$?
- Why do we sort for Merge Intervals?
- What invariant is maintained while merging intervals?
- What is the complexity of Merge Intervals?
- How would you solve interval intersection?
- When would you use a Hash Map instead of a Hash Set?

PART 8 — GOOGLE INTERVIEW PREP REPOSITORY

Your current repository has a day5/ directory but no visible day6/ yet.

Do not overwrite Day 5.

Create day6/ only if you have not already committed your Day 6 work locally and simply haven't pushed it.

Before creating anything:

```
git status
git log --oneline --all --decorate -n 15
```

Then reconcile the local state with GitHub.

Day 7 structure:

```
day7/
├── dsa/
│   ├── hashing-pattern.md
│   ├── group-anagrams.md
│   ├── longest-consecutive-sequence.md
│   ├── merge-intervals.md
│   └── insert-interval.md
├── backend/
│   ├── database-failures.md
│   ├── connection-pooling.md
│   ├── transactions-and-rollback.md
│   └── optimistic-concurrency.md
├── system-design/
│   └── document-service-scale.md
├── interview/
│   └── day7-interview-questions.md
└── README.md
```

Only create the files for work you actually complete.

PART 9 — PROJECT 1 DOCUMENTATION

The current README already documents:

- architecture
- schema

- configuration
- migrations
- connection lifecycle
- exception handling
- CRUD
- PostgreSQL behavior
- integration tests

Therefore do not rewrite the README from scratch.

Add a small Day 7 section:

```
## Reliability and Failure Handling
```

Document:

```
Database failure behavior
Connection cleanup
Rollback behavior
SQL parameterization
Concurrency considerations
Known limitations
```

Especially document things that are not yet implemented.

For example, the current README explicitly says that database-level CHECK constraints and separate test-database provisioning have not been expanded beyond the current scope.

That is exactly the kind of honest engineering documentation we want.

PART 10 — GIT

Do not create arbitrary commits just to have commits.

Your actual work should determine them.

A reasonable Day 7 sequence might be:

```
test: add database failure coverage
refactor: harden database resource handling
test: cover repository integrity failures
docs: document backend failure behavior
```

But only use these if those are actually the changes you make.

DAY 7 — EXACT EXECUTION ORDER

00:00–00:10 — Baseline

- git status
- inspect latest commits
- run full pytest suite
- record actual result
- inspect current architecture
- identify any existing failure/warning

00:10–01:00 — DSA

- Group Anagrams
- Longest Consecutive Sequence
- Merge Intervals
- Insert Interval only if time permits
- write pattern-recognition clues
- write complexity
- write edge cases
- answer follow-ups

01:00–01:30 — Backend theory

- database failure
- transaction rollback
- connection lifecycle
- connection pooling
- SQL parameterization
- concurrency
- lost update
- optimistic concurrency

01:30–02:30 — Project hardening

- inspect `connection.py`
- inspect repository error paths
- inspect service/repository boundaries
- inspect SQL parameterization
- inspect resource cleanup
- identify one genuine weakness
- implement one meaningful hardening improvement
- avoid unnecessary refactoring

02:30–02:55 — Testing

- full test suite
- database failure behavior
- missing document
- delete missing document
- duplicate/integrity behavior
- inspect test isolation
- record actual results

02:55–03:05 — System design + documentation

- 10× document-service design
- update interview-prep notes
- update Project 1 reliability documentation
- commit actual work
- push

If you have only three hours, stop at 3 hours. Do not start another feature.

DAY 7 DEFINITION OF DONE

DSA

- Group Anagrams solved independently
- Longest Consecutive Sequence solved independently
- Merge Intervals solved independently
- Interval pattern understood
- Hashing pattern understood
- Complexity explained without notes
- Edge cases explained
- Pattern recognition notes committed

Backend

- Explain complete API request lifecycle
- Explain DB connection lifecycle
- Explain rollback
- Explain connection pooling
- Explain SQL parameterization
- Explain lost updates
- Explain optimistic concurrency
- Explain database failure behavior

Project 1

- Current code inspected before modification
- Existing tests pass before changes
- One genuine reliability weakness identified
- Meaningful hardening implemented
- No unnecessary architecture rewrite
- SQL reviewed for parameterization
- resource cleanup reviewed
- failure path tested

- existing CRUD contract preserved

Testing

- happy paths still pass
- failure paths tested
- DB integrity behavior understood
- test isolation verified
- actual pytest result recorded

System Design

- 10× architecture designed
- scaling bottleneck identified
- database bottleneck identified
- reliability strategy identified
- observability strategy identified

Git

- only real work committed
- no secrets
- README/docs reflect actual state
- pushed to GitHub

MOST IMPORTANT: WHAT DAY 7 IS NOT

- ■ Redis
- ■ Docker
- ■ Kubernetes
- ■ GCP
- ■ React
- ■ RAG
- ■ embeddings
- ■ vector database
- ■ agents
- ■ MLflow
- ■ Project 3
- ■ PDF ingestion
- ■ microservices
- ■ another PostgreSQL CRUD implementation
- ■ another Prefix Sum tutorial

Your Project 1 already has PostgreSQL persistence, migrations, connection management, and integration tests.

The correct progression now is:

```
DAY 1-4
Python
FastAPI
Pydantic
Layered Backend
Testing
Exception Handling
↓
DAY 5
PostgreSQL
Migrations
Persistent Repository
Integration Testing
↓
DAY 6
Prefix Sum + Hashing
Algorithm comparison
Pattern recognition
↓
DAY 7
Hashing + Intervals
+
Backend Failure/Concurrency
+
Reliability Hardening
↓
NEXT
Deeper Backend / SWE fundamentals
↓
ML
↓
RAG
↓
Agents
↓
Full Stack
↓
Kubernetes / GCP
↓
Production AI
```

The key change in strategy

Day 5 was about making the system persistent.

Day 6 was about expanding your algorithmic pattern vocabulary.

Day 7 is about making you think like the engineer responsible for that persistent system when things go wrong.

That is the correct next step for the Google-level Software Engineer first → AI/ML specialization profile you're building.